

Title

**Computer Vision Application for Image
Enhancement and Geometrical Transformations
Using Python**

Submitted By

Name: YASH RAJ SINGH

Registration Number:

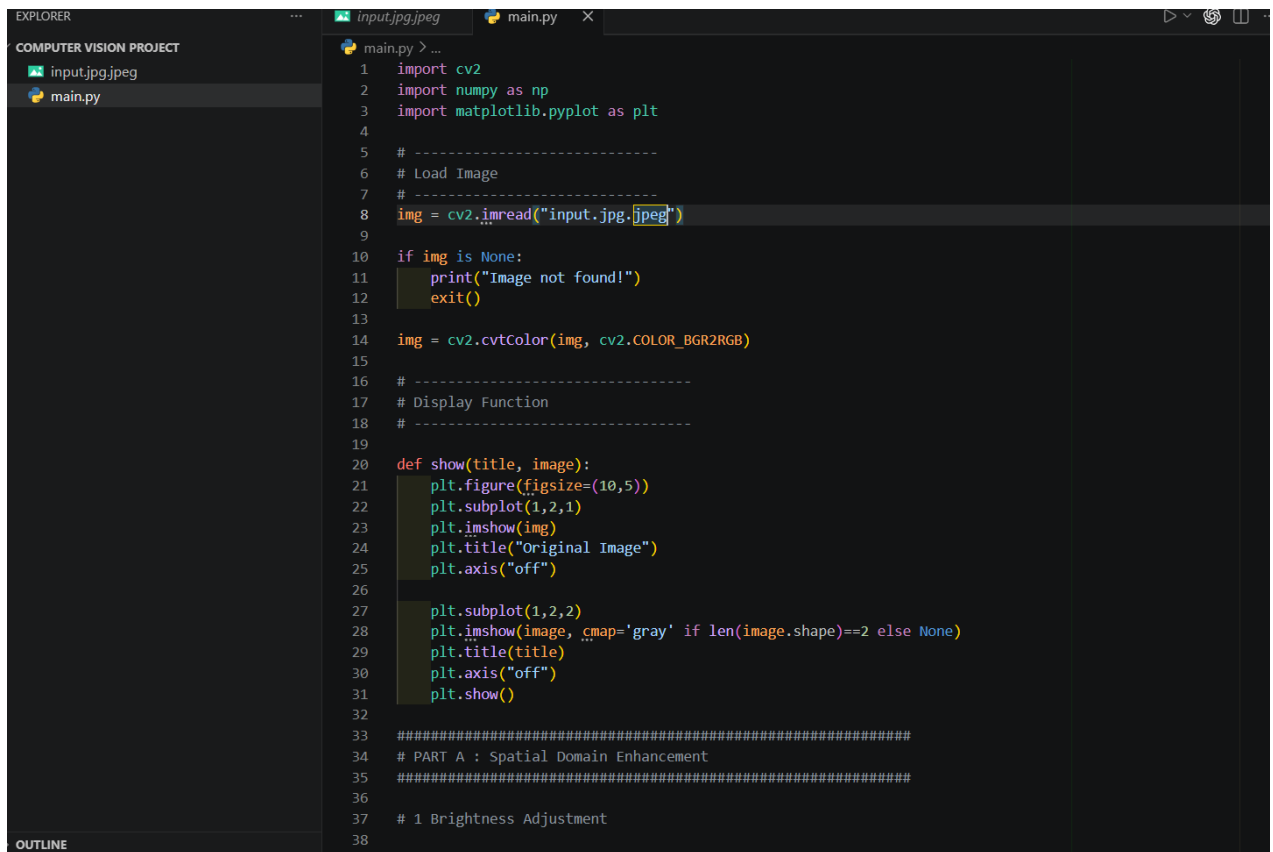
24BAI10569

Course: BTECH CSE(AIML)

Subject: Computer Vision

Date:21 JULY,2026

PROJECT STRUCTURE IN VS CODE



```
EXPLORER
COMPUTER VISION PROJECT
  input.jpg.jpeg
  main.py

main.py > ...
1  import cv2
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  # -----
6  # Load Image
7  # -----
8  img = cv2.imread("input.jpg.jpeg")
9
10 if img is None:
11     print("Image not found!")
12     exit()
13
14 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
15
16 # -----
17 # Display Function
18 # -----
19
20 def show(title, image):
21     plt.figure(figsize=(10,5))
22     plt.subplot(1,2,1)
23     plt.imshow(img)
24     plt.title("Original Image")
25     plt.axis("off")
26
27     plt.subplot(1,2,2)
28     plt.imshow(image, cmap='gray' if len(image.shape)==2 else None)
29     plt.title(title)
30     plt.axis("off")
31     plt.show()
32
33 #####
34 # PART A : Spatial Domain Enhancement
35 #####
36
37 # 1 Brightness Adjustment
38
```

COMPLETE CODE FOR ALL THE OPERATIONS-

```
main.py > ...
1  import cv2
2  import numpy as np
3  import matplotlib.pyplot as plt
4
5  # -----
6  # Load Image
7  # -----
8  img = cv2.imread("input.jpg.jpeg")
9
10 if img is None:
11     print("Image not found!")
12     exit()
13
14 img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
15
16 # -----
17 # Display Function
18 # -----
19
20 def show(title, image):
21     plt.figure(figsize=(10,5))
22     plt.subplot(1,2,1)
23     plt.imshow(img)
24     plt.title("Original Image")
25     plt.axis("off")
26
27     plt.subplot(1,2,2)
28     plt.imshow(image, cmap='gray' if len(image.shape)==2 else None)
29     plt.title(title)
30     plt.axis("off")
31     plt.show()
32
33 #####
34 # PART A : Spatial Domain Enhancement
35 #####
36
37 # 1 Brightness Adjustment
38
39 brightness = 50
```

```
38
39     brightness = 50
40
41     bright = cv2.convertScaleAbs(img, alpha=1, beta=brightness)
42
43     show("Brightness Adjustment", bright)
44
45
46     # 2 Contrast Adjustment
47
48     contrast = 1.5
49
50     contrast_img = cv2.convertScaleAbs(img, alpha=contrast, beta=0)
51
52     show("Contrast Adjustment", contrast_img)
53
54
55     # 3 Image Negative
56
57     negative = 255 - img
58
59     show("Image Negative", negative)
60
61
62     # 4 Log Transformation
63
64     c = 255 / np.log(1 + np.max(img))
65
66     log_image = c * np.log(1 + img.astype(np.float32))
67
68     log_image = np.array(log_image, dtype=np.uint8)
69
70     show("Log Transformation", log_image)
71
72
73     # 5 Gamma Transformation
74
75     gamma = 2.0
76
77     gamma_corrected = np.array(255*(img/255)**gamma, dtype='uint8')
```

```

79  show("Gamma Transformation", gamma_corrected)
80
81
82  # 6 Thresholding
83
84  gray = cv2.cvtColor(img,cv2.COLOR_RGB2GRAY)
85
86  threshold_value = 127
87
88  _,thresh=cv2.threshold(gray,threshold_value,255,cv2.THRESH_BINARY)
89
90  show("Thresholding",thresh)
91
92
93  # 7 Contrast Stretching
94
95  min_val=np.min(img)
96  max_val=np.max(img)
97
98  stretch=((img-min_val)/(max_val-min_val)*255).astype(np.uint8)
99
100 show("Contrast Stretching",stretch)
101
102 #####
103 # PART B : Geometrical Transformations
104 #####
105
106 rows,cols=img.shape[:2]
107
108 # 1 Translation
109
110 tx=100
111 ty=50
112
113 M=np.float32([[1,0,tx],[0,1,ty]])
114
115 translated=cv2.warpAffine(img,M,(cols,rows))
116
117 show("translation", translated)

```

```
115 translated=cv2.warpAffine(img,M,(cols,rows))
116
117 show("Translation",translated)
118
119
120 # 2 Scaling
121
122 scale=1.5
123
124 scaled=cv2.resize(img,None,fx=scale,fy=scale)
125
126 show("Scaling",scaled)
127
128
129 # 3 Rotation
130
131 angle=45
132
133 M=cv2.getRotationMatrix2D((cols/2,rows/2),angle,1)
134
135 rotated=cv2.warpAffine(img,M,(cols,rows))
136
137 show("Rotation",rotated)
138
139
140 # 4 Horizontal Reflection
141
142 horizontal=cv2.flip(img,1)
143
144 show("Horizontal Reflection",horizontal)
145
146
147 # 5 Vertical Reflection
148
149 vertical=cv2.flip(img,0)
150
151 show("Vertical Reflection",vertical)
152
153
154 # 6 Reflection about Origin
```

```

154 # 6 Reflection about Origin
155
156 origin=cv2.flip(img,-1)
157
158 show("Reflection about Origin",origin)
159
160
161 # 7 Shearing X-axis
162
163 shear_x=0.5
164
165 M=np.float32([[1,shear_x,0],
166              [0,1,0]])
167
168 sheared_x=cv2.warpAffine(img,M,(int(cols+rows*shear_x),rows))
169
170 show("Shearing X-axis",sheared_x)
171
172
173 # 8 Shearing Y-axis
174
175 shear_y=0.5
176
177 M=np.float32([[1,0,0],
178              [shear_y,1,0]])
179
180 sheared_y=cv2.warpAffine(img,M,(cols,int(rows+cols*shear_y)))
181
182 show("Shearing Y-axis",sheared_y)
183
184
185 # 9 Affine Transformation
186
187 pts1=np.float32([[50,50],[200,50],[50,200]])
188
189 pts2=np.float32([[10,100],[200,50],[100,250]])
190
191 M=cv2.getAffineTransform(pts1,pts2)

```

```
161 # 7 Shearing X-axis
162
163 shear_x=0.5
164
165 M=np.float32([[1,shear_x,0],
166               [0,1,0]])
167
168 sheared_x=cv2.warpAffine(img,M,(int(cols+rows*shear_x),rows))
169
170 show("Shearing X-axis",sheared_x)
171
172
173 # 8 Shearing Y-axis
174
175 shear_y=0.5
176
177 M=np.float32([[1,0,0],
178               [shear_y,1,0]])
179
180 sheared_y=cv2.warpAffine(img,M,(cols,int(rows+cols*shear_y)))
181
182 show("Shearing Y-axis",sheared_y)
183
184
185 # 9 Affine Transformation
186
187 pts1=np.float32([[50,50],[200,50],[50,200]])
188
189 pts2=np.float32([[10,100],[200,50],[100,250]])
190
191 M=cv2.getAffineTransform(pts1,pts2)
192
193 affine=cv2.warpAffine(img,M,(cols,rows))
194
195 show("Affine Transformation",affine)
```

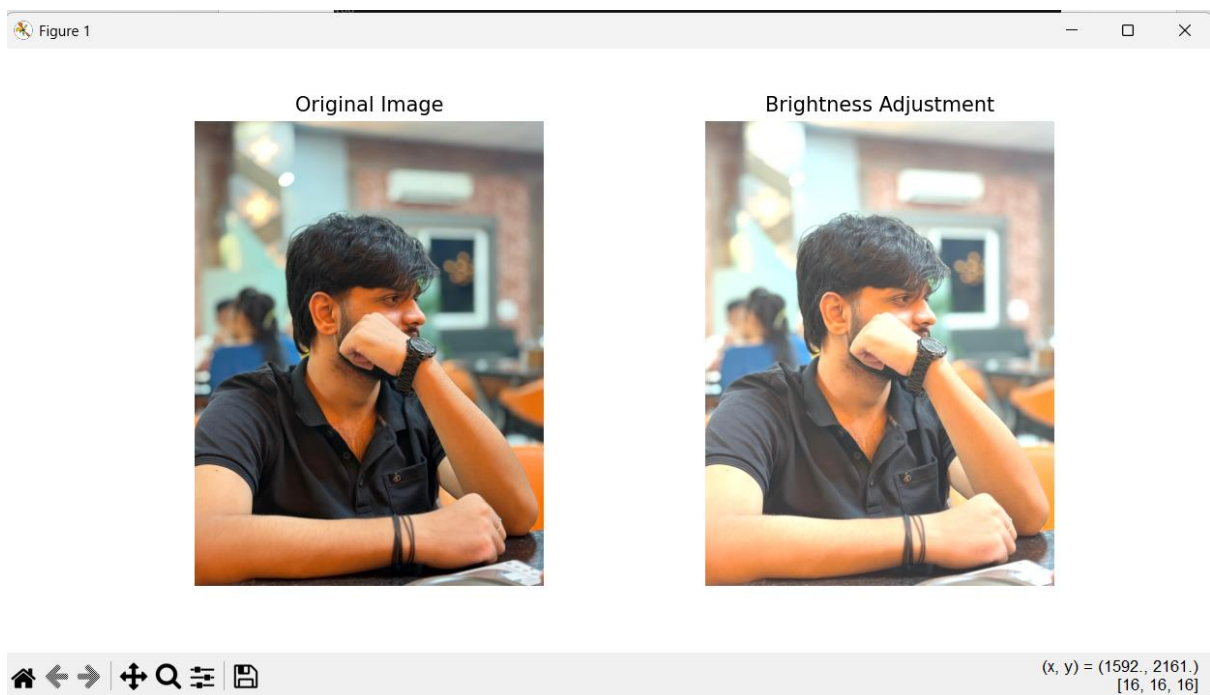
INPUT IMAGE-



OUTPUT-

ORIGINAL IMAGE

COORDINATES-



PART A: Spatial Domain Enhancement

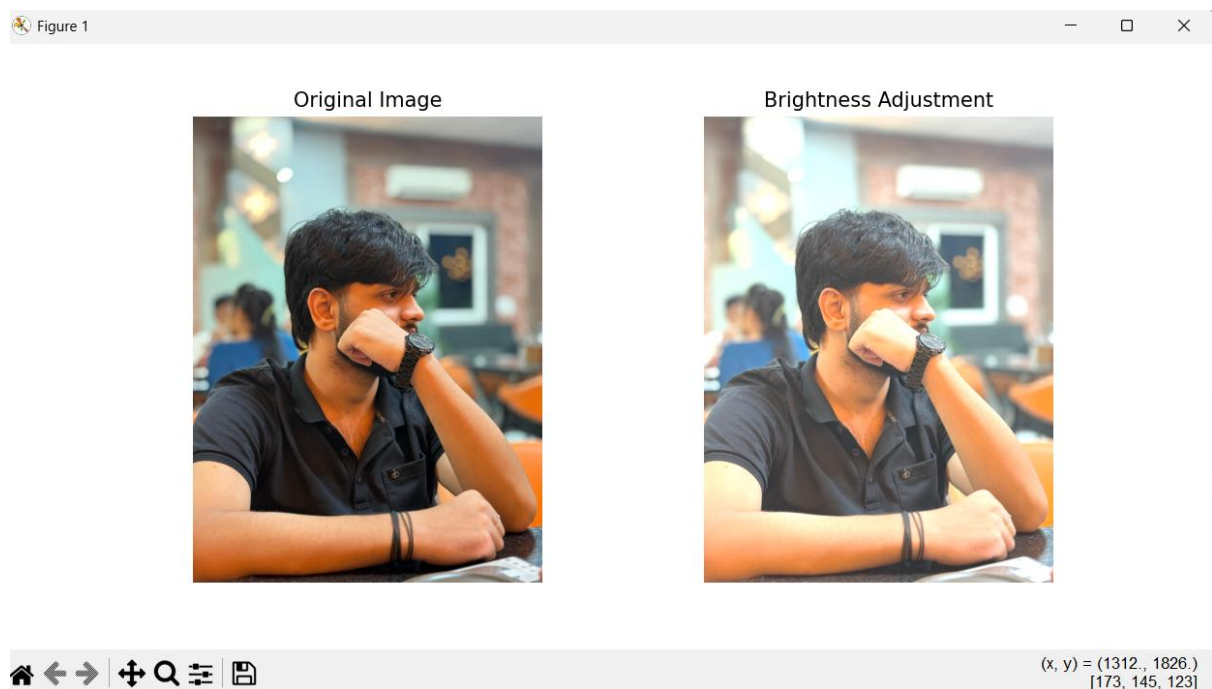
1. Brightness Adjustment

Description

Brightness adjustment increases or decreases the intensity of all pixels, making the image brighter or darker.

Parameter Used

Brightness Value (β) = 50



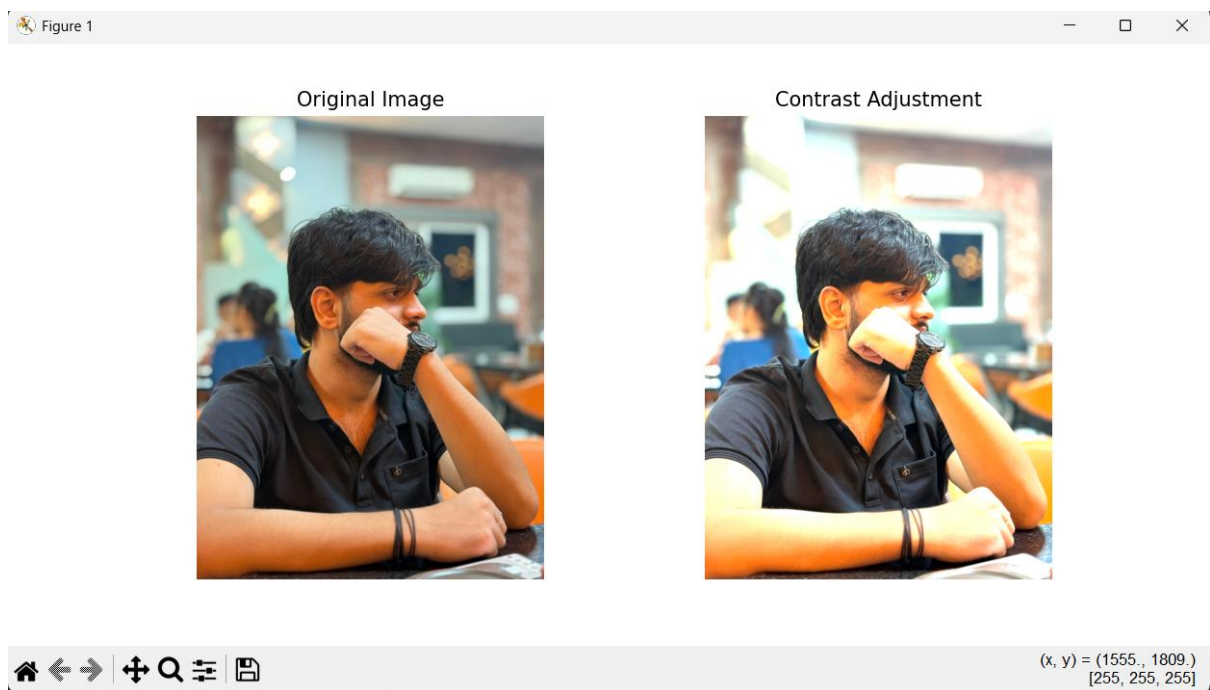
2. Contrast Adjustment

Description

Contrast adjustment increases the difference between dark and bright regions.

Parameter Used

Contrast Factor (α) = 1.5



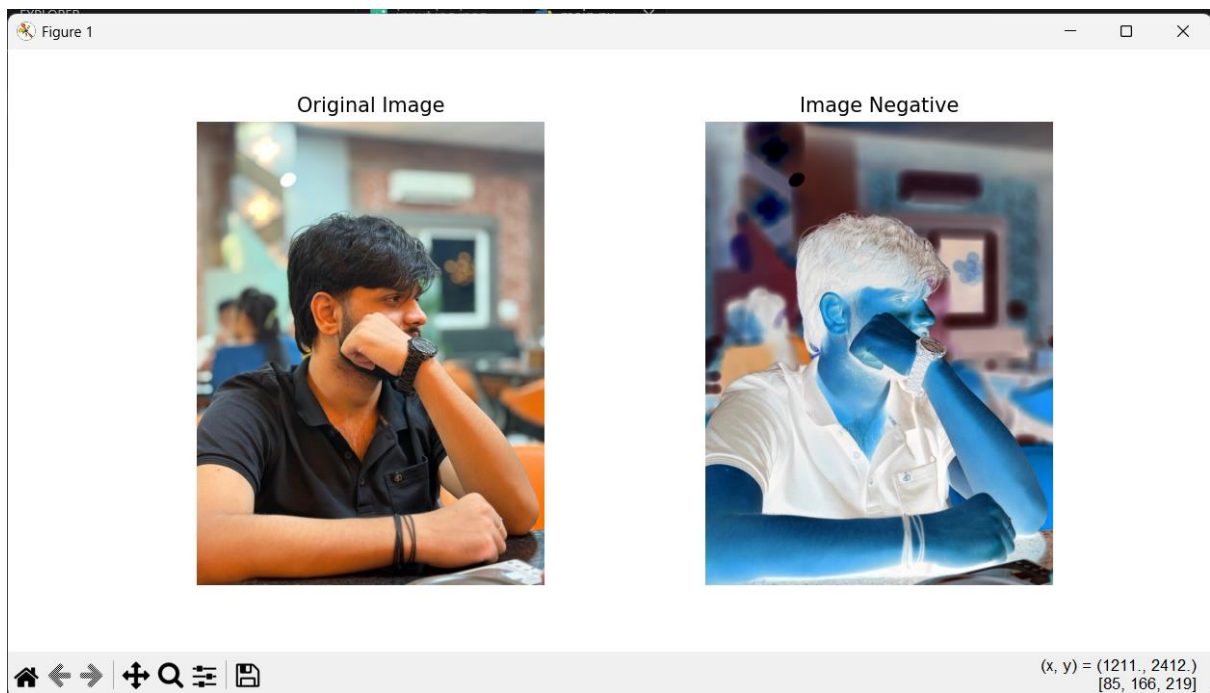
3. Image Negative

Description

Image negative inverts the pixel intensity values.

Formula

Negative = 255 – Pixel Value



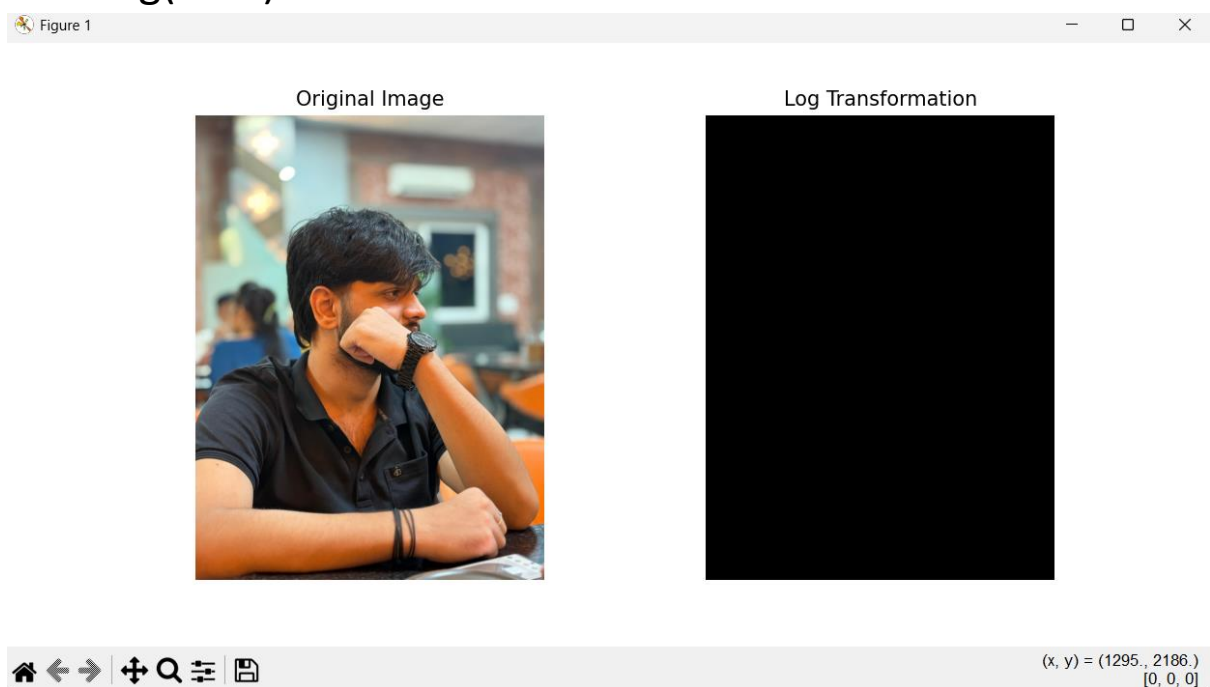
4. Log Transformation

Description

Log transformation enhances darker regions while compressing brighter regions.

Formula

$$s = c \log(1 + r)$$



5. Gamma Transformation

Description

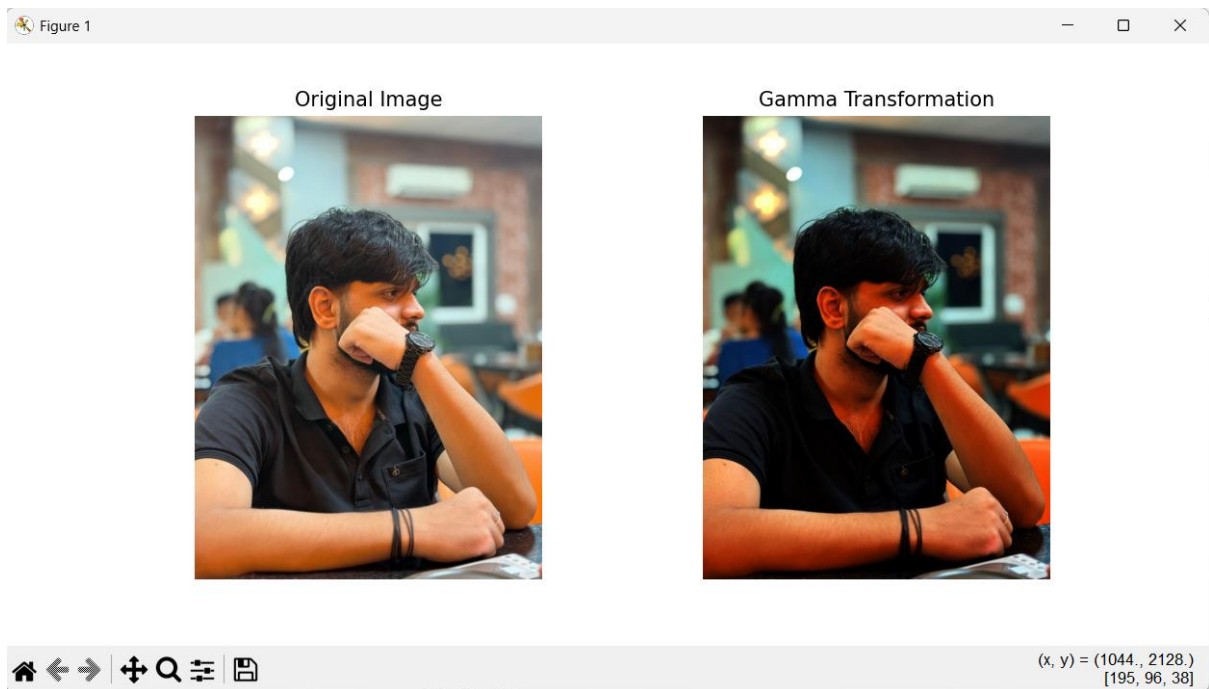
Gamma correction controls image brightness using power-law transformation.

Formula

$$s = c r^{\gamma}$$

Parameter

Gamma = **2.2**



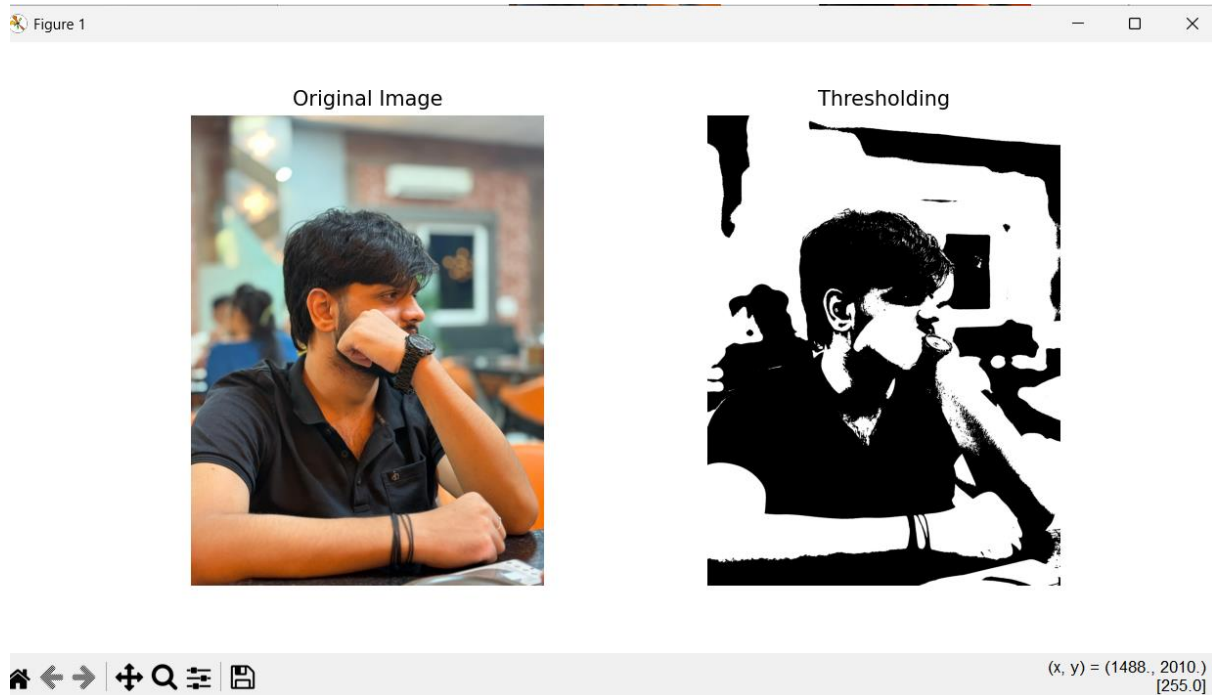
6. Thresholding

Description

Thresholding converts a grayscale image into a binary image.

Parameter

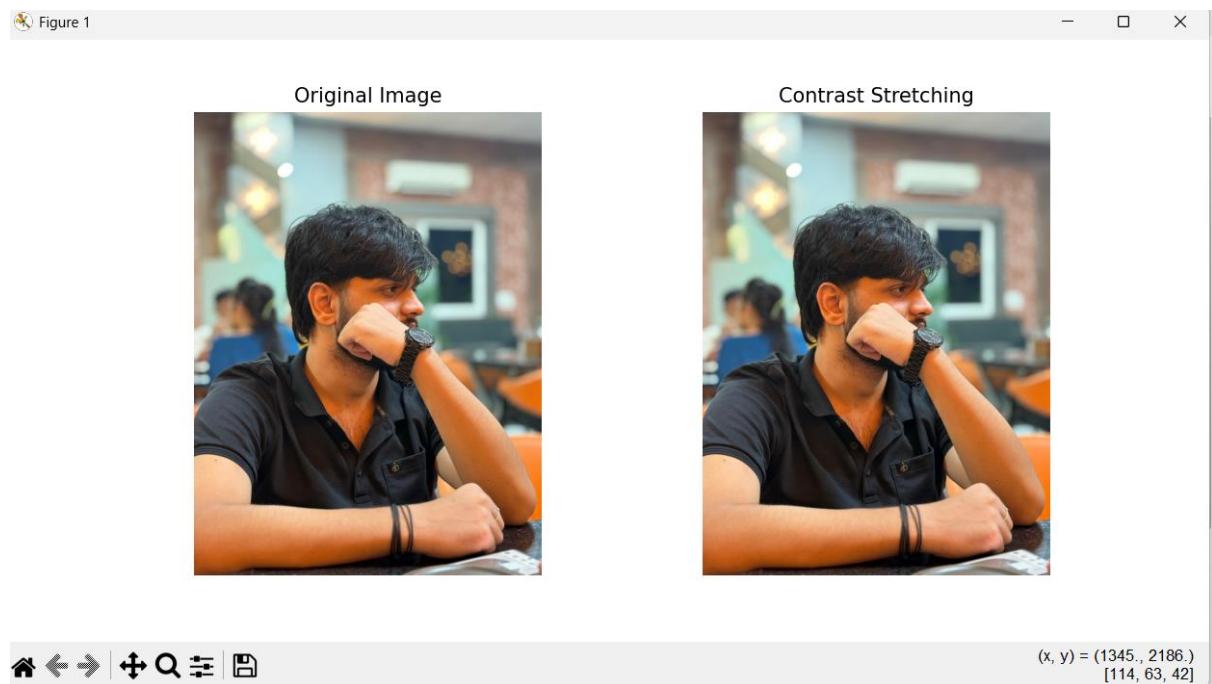
Threshold Value = **127**



7. Contrast Stretching

Description

Contrast stretching increases the dynamic range of pixel values.



PART B: Geometrical Transformations

1. Translation

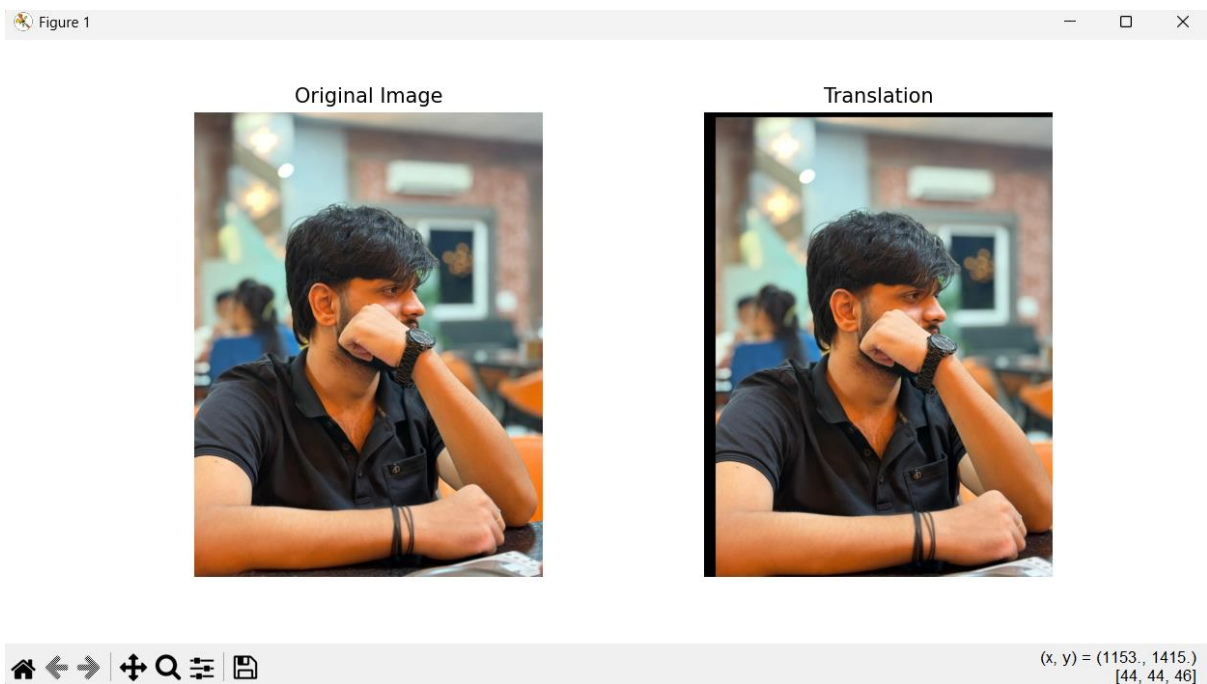
Description

Translation shifts the image horizontally and vertically.

Parameters

$t_x = 100$

$t_y = 50$



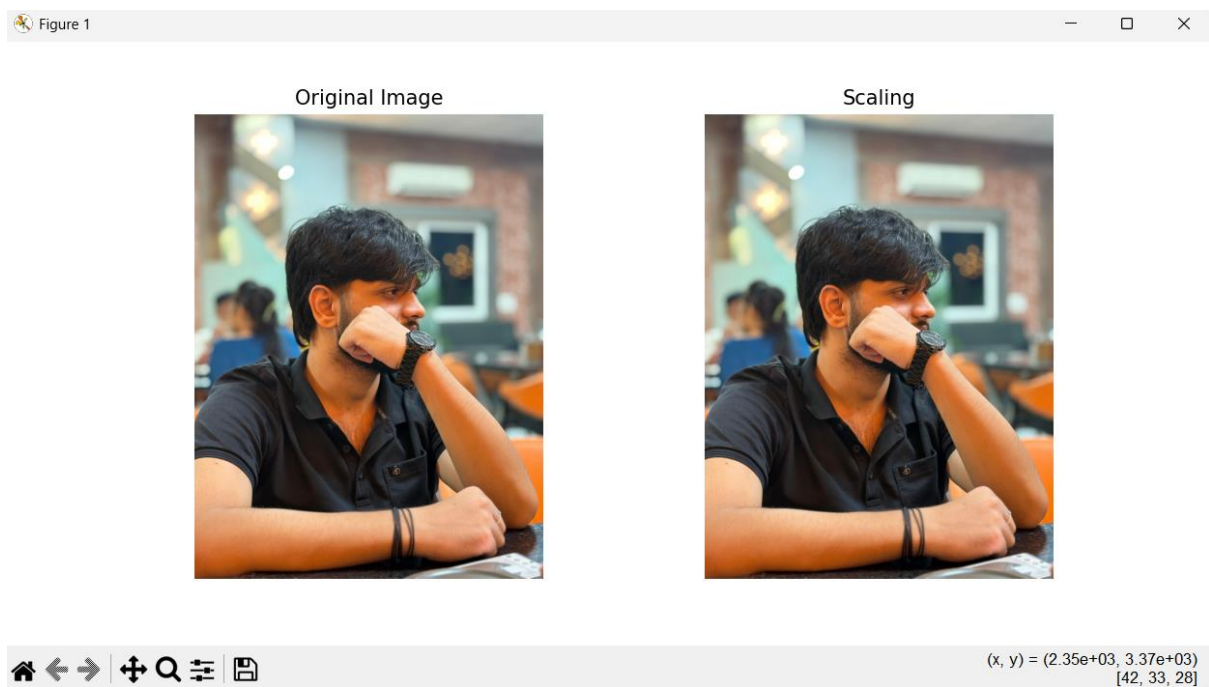
2. Scaling

Description

Scaling changes the size of the image.

Parameter

Scale = 1.5



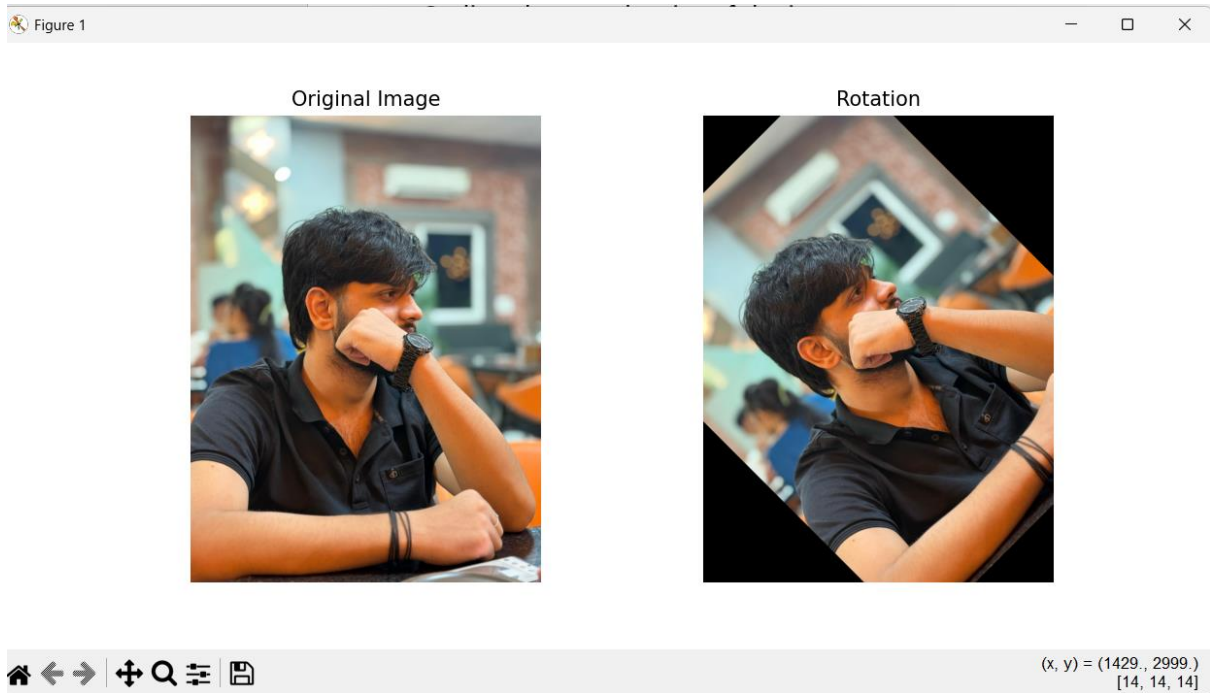
3. Rotation

Description

Rotation rotates the image around its center.

Parameter

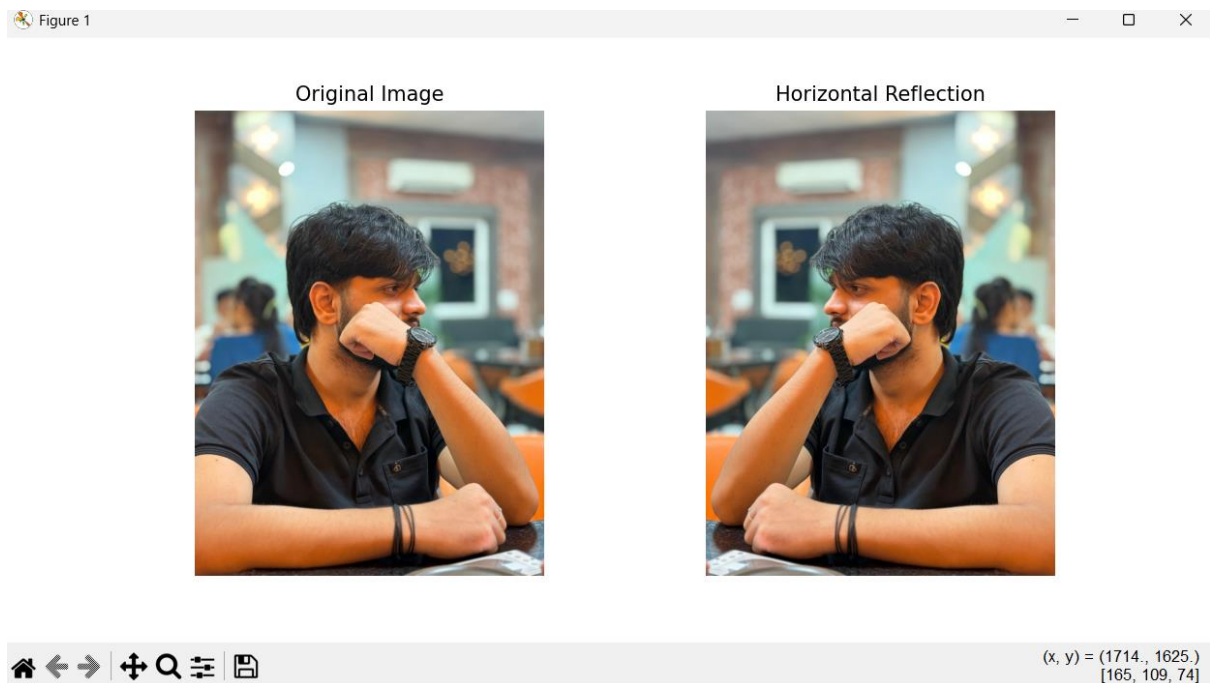
45°



4. Horizontal Reflection

Description

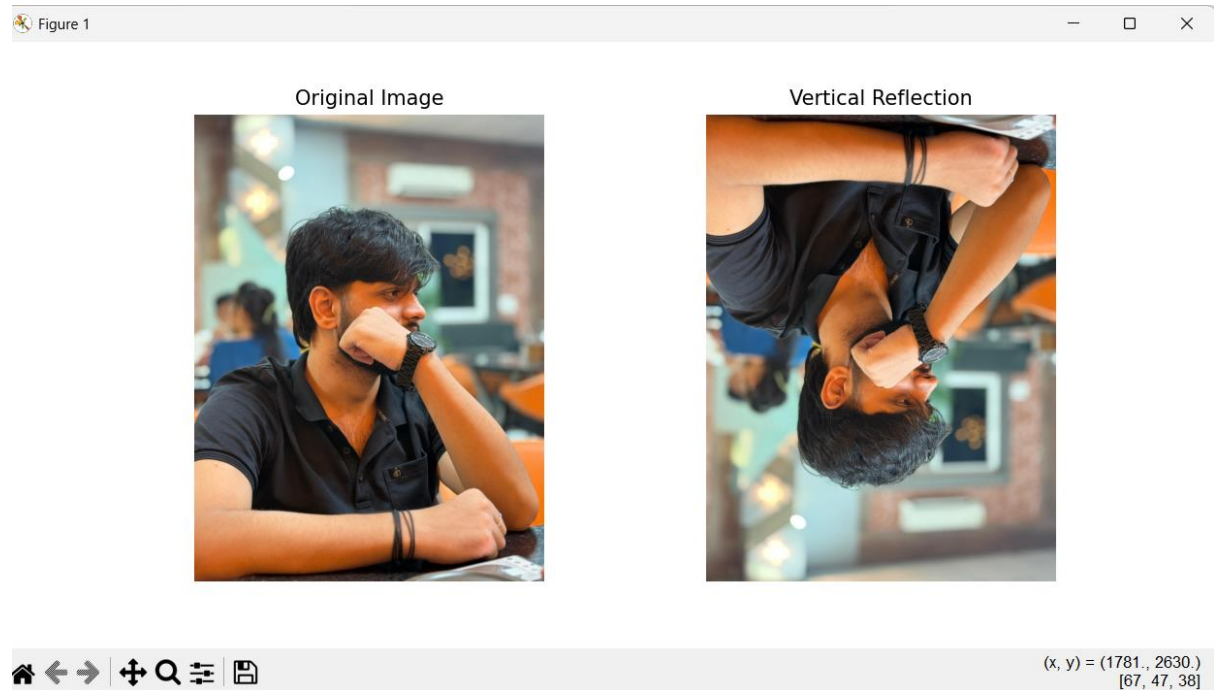
Flips the image from left to right.



5. Vertical Reflection

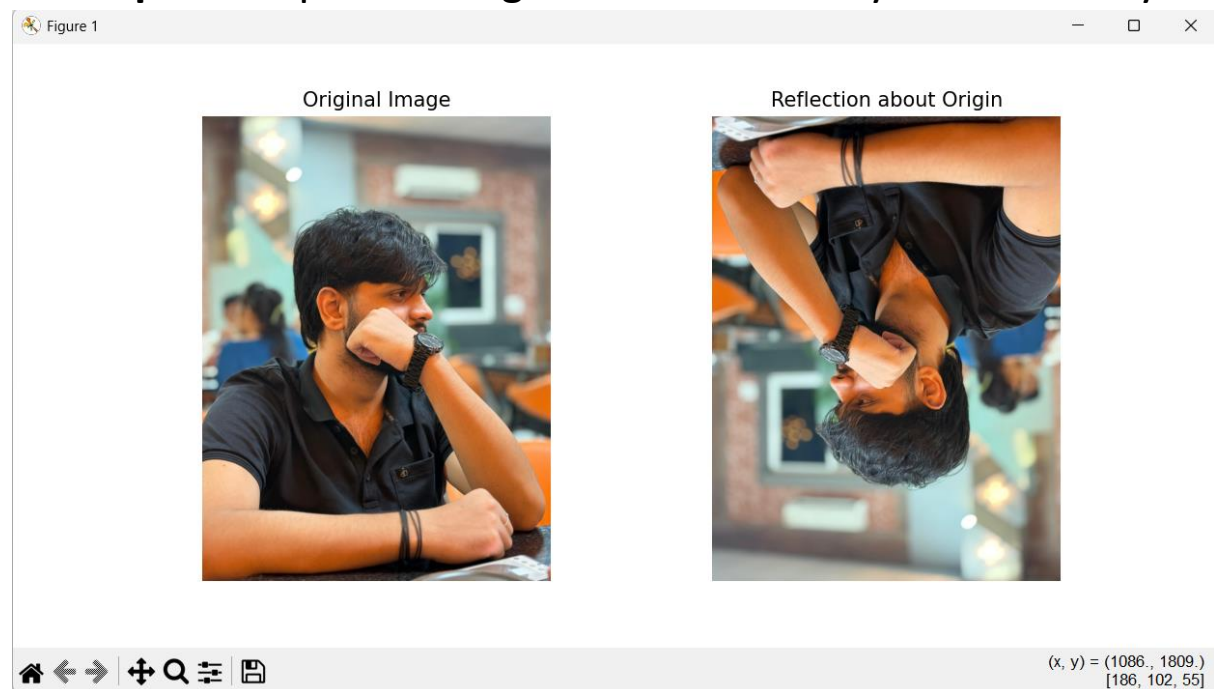
Description

Flips the image upside down.



6. Reflection About Origin

Description-Flips the image both horizontally and vertically



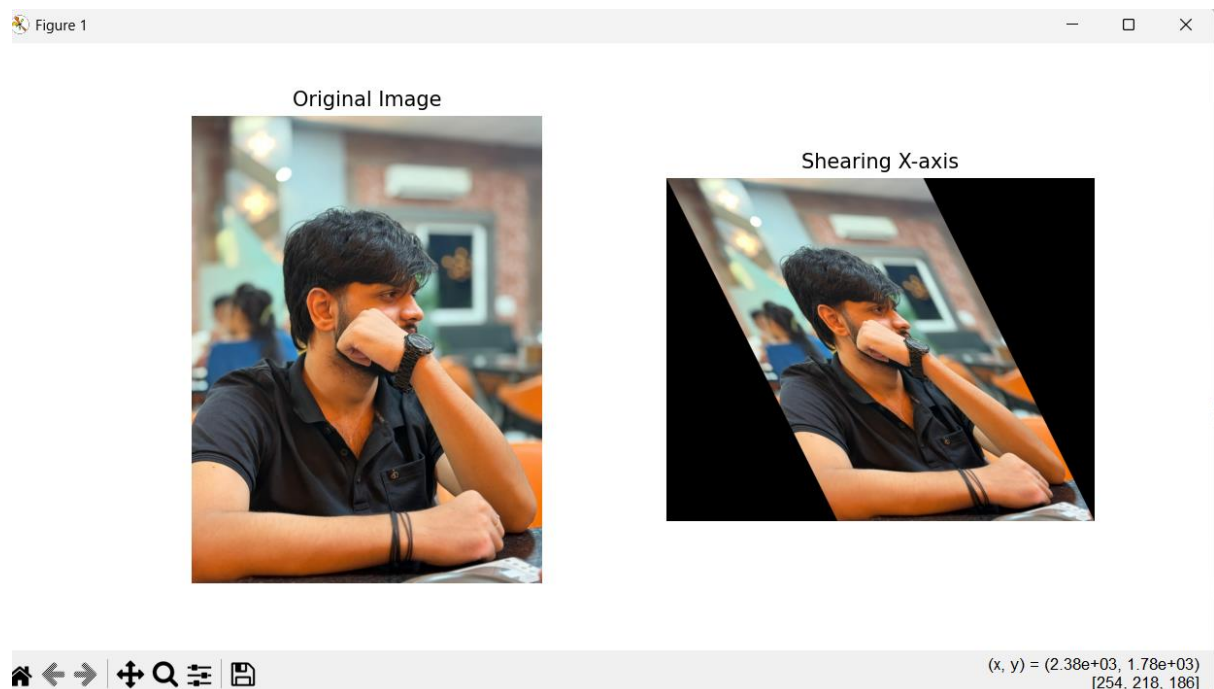
7. Shearing Along X-axis

Description

Shears the image along the horizontal direction.

Parameter

0.5



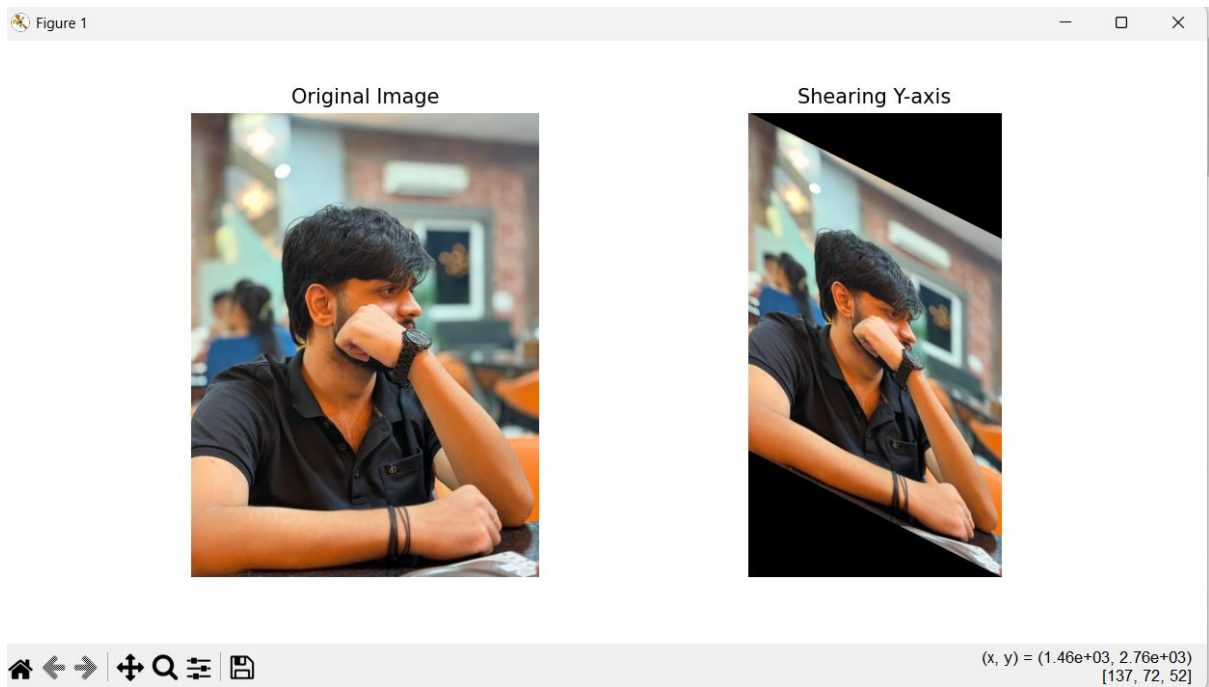
8. Shearing Along Y-axis

Description

Shears the image along the vertical direction.

Parameter

0.5



9. Affine Transformation

Description

Affine transformation preserves straight lines while applying translation, rotation, scaling, and shearing.

Output

